SNUG EUROPE 2005

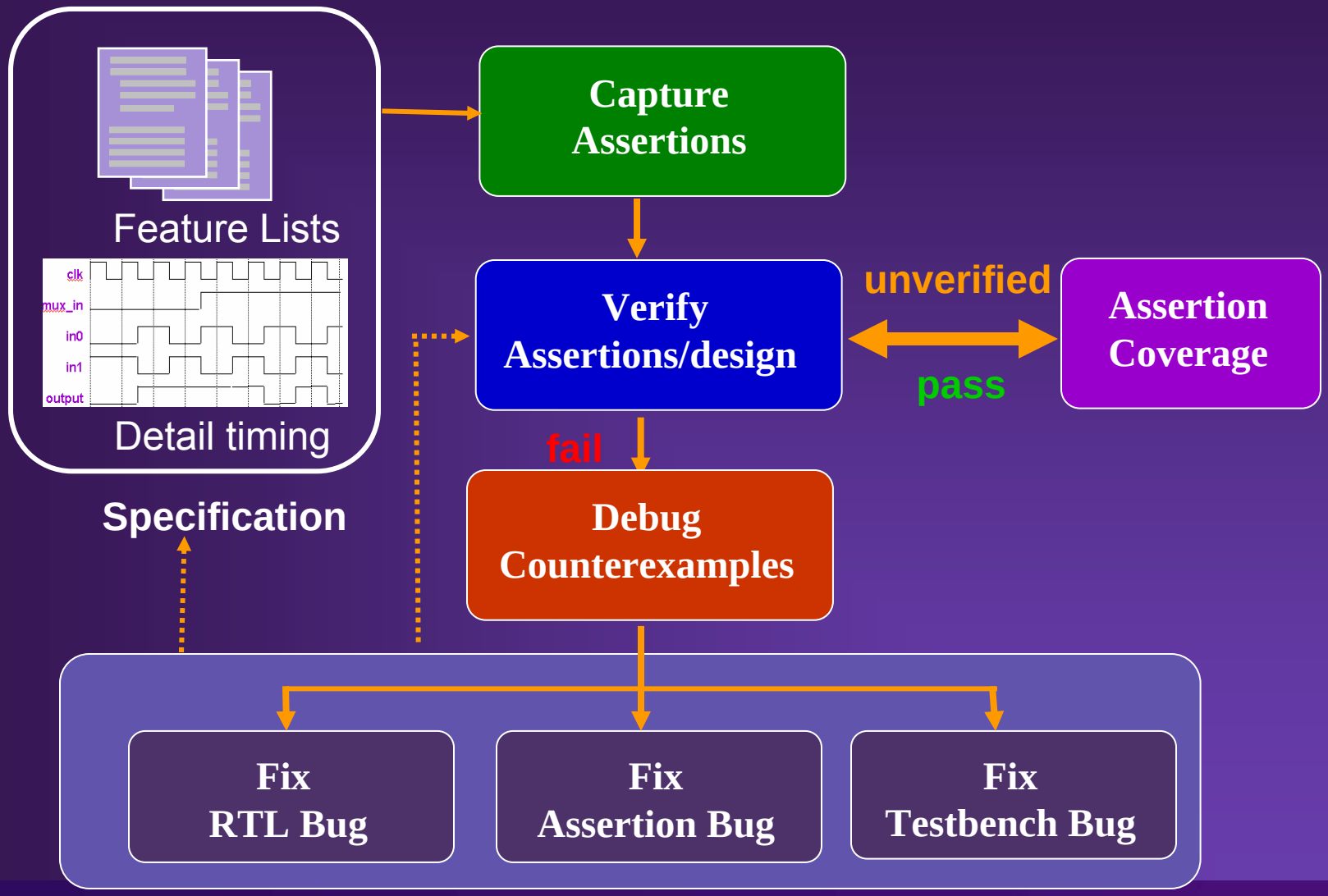
Best Practices of Assertion Based Verification using SystemVerilog Assertions, VCS and Magellan

Presented by:
Jin Hou
John Girard

Agenda

Unit	Topic
1	Assertion Methodology
2	Design Under Test Overview
3	Design Under Test Assertion Examples
4	Assertion Coding Guidelines

Assertion Based Verification Flow



SYNOPSYS®

Capturing Assertions

- Well defined specification will save time
  - Poor documentation causes undesirable behaviors
  - Detailed timing and descriptions for signal interactions are a must

- Use powerful assertion language SVA
- Leverage assertion IP libraries
  - Will save time (alleviate frustration)
  - Pre-verified (high-level behaviors)
  - Encapsulate all low level details (assertion code)
  - Increases assertion density

Verifying Assertions

- **A combination of algorithms and engines is used to provide a verification result**
  - **Simulation environment:**
    - Demonstrates checker functionality
    - Verifies that illegal behavior is trapped via bug injection

  - **Pure formal environment:**
    - Static model checker to find proofs & bugs
    - Some properties cannot be completely verified
  - **Hybrid formal environment:**
    - Combination of formal and simulation to find bugs
    - Much higher capacity than pure formal
    - Better state-space coverage than pure simulation

Assertion Coverage

- Identifies deficiencies (holes) in your testing
  - Did the test suite properly exercise each assertion?
    - Attempt succeed or failed
  - Were there any vacuous (only) success or proofs?
    - Antecedents that never evaluated true

```
A |-> B;
```

  - Did every path in the assertion get exercised
    - Multiple branches to successful completion

```
A |-> ##[0..10] (B || C) ;
```

variable range

completes if either B or C is true

SNUG EUROPE 2005

SYNOPSYS®

Debugging Counterexamples

- Counterexample:
  - Input (dynamic) sequence that demonstrates violations of an assertion
- Property violations occur when:
  - Assertion code is wrong
  - Input sequence may not be constrained properly
  - There is a bug in the design
- Violations need to be visualized
  - Text based or Graphical analysis

Why SystemVerilog Assertions

- Describes system functions accurately and concisely
- Provides rich operators for sequences and properties
  - Sequence: cycle delay, repetition, and, or, intersect, within, first_match, throughout, ended, matched, local variables
  - Property: disable iff, not, implication, nested, recursive
- Supports single and multiple clocks
- Supports assertions: concurrent, procedural and immediate
- Supports functional coverage and data coverage
- Supports reactive testbenches (action blocks and task invocation)
- Has rich SVA library
- Can be used for dynamic and formal verification

Agenda

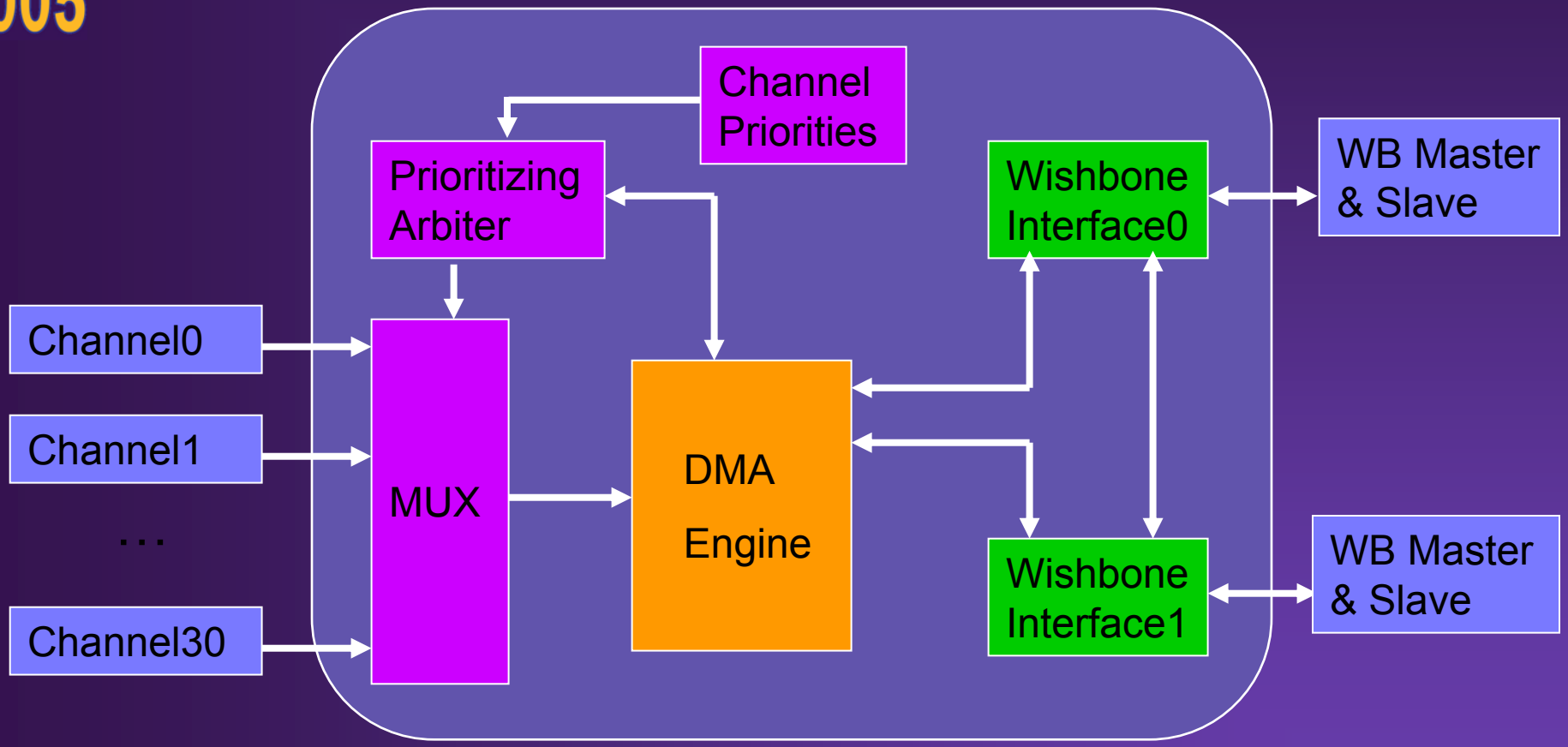
Unit	Topic
1	Assertion Methodology
2	Design Under Test Overview
3	Design Under Test Assertion Examples
4	Assertion Coding Guidelines

Project Objectives

- Pick a reference design that contains typical RTL design structures
- Develop properties that describe how the design is intended to operate
- Define assertions based on the properties
- Use Synopsys VCS and Magellan to verify the assertions

Reference DUT Description

Wishbone DMA/Bridge Core



available @ http://www.opencores.org/cvsweb.shtml/wb_dma/

DMA Engine Overview

(DMA/Bridge)

- DMA engine can perform several types of transfers
  - Normal (Software) DMA mode
    - DMA engine performs block copies
    - Transfer begins when local controller/CPU writes to channel CRS register
    - DMA engine goes to idle when transfer finishes
  - HW handshake Mode
    - External request DMA_REQ_I triggers DMA transfer
    - DMA engine asserts DMA_ACK_O upon completion of a chunk of words
    - DMA engine asserts DONE when it completes the transfer
  - Pass through mode
    - DMA doesn't add any functionality to path-through traffic

Wishbone Interface Overview

(DMA/Bridge)

- **MASTER / SLAVE architecture**
  - All bus cycles use handshaking protocol between the MASTER and SLAVE
- **Data transfer types**
  - READ/WRITE cycle
  - BLOCK transfer cycle
  - Single and multiple clock data transfers
  - Interruptible (no lock out)
- **Supports standard interfacing only**
  - No user defined signals (Address and Data tags)

DMA Arbiter Overview

(DMA/Bridge)

- 31 input (requests) arbiter
  - Supports up to 8 priorities
- Round-robin arbitration algorithm for DMA channels of same priority
  - Switches grant if current *req* is dropped or *next* is asserted
  - Parks at last *grant*

Verification Strategy

Black Box

- Focus on functional requirements (Black Box)
  - Write assertions to cover as many functionalities in the SPEC
  - Encapsulate each interface with a set of SVA
- Attempt to find:
  - Interface errors (protocol)
  - Specification errors
  - Performance or throughput errors

Verification Strategy

White Box

- Focus on RTL design structures (White Box)
  - Write assertions (whenever possible) to test internal logic to ensure validity
- Attempt to find:
  - FSM errors
  - Arbitration errors
  - Encoding errors
  - Overflow/Underflow errors

Magellan Tool

- Magellan is both a formal and hybrid engine
  - Deploys formal algorithms to prove properties or find unreachable goals
  - Uses hybrid algorithm (combining simulation and formal) to find property falsifications or reachable goals
- Key benefits:
  - We could immediately test and debug the RTL and assertions
    - We didn't have to wait until a testbench was available
    - Easily created replay traces for identifying and debugging falsified properties
    - High capacity for large designs

Test Plan

- **Develop custom assertions**
  - **Use assertion libraries whenever possible**
    - ***assert_arbiter*** **(Priority Arbiter)**
    - ***assert_no_underflow*** **(chunk & Size counters)**
- **Run DMA/Bridge IP regression suite**
  - **Debug failing properties**
  - **Collect coverage information (Line and Assertion)**

Test Plan Cont'd

- Run hybrid formal RTL verification
  - Check the correctness of the design (proofs)
  - Debug any failing properties (falsifications)
  - Check for vacuous proofs (reachable)
- Find the areas that need more properties
  - Magellan reports the circuit uncovered by the property cone influence circuit
  - Collects code coverage when running property checking
  - Implement more properties for the low coverage areas

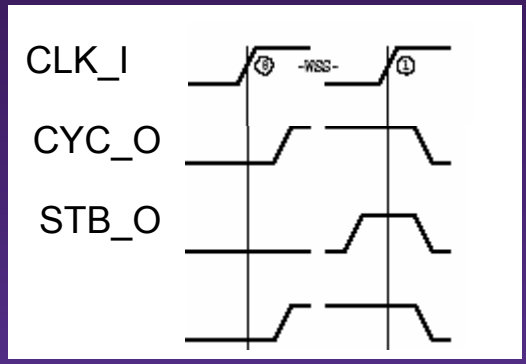
Agenda

Unit	Topic
1	Assertion Methodology
2	Design Under Test Overview
3	Design Under Test Assertion Examples
4	Assertion Coding Guidelines

Check Wishbone Master

WB_MAS_3_25

- **RULE 3.25**
  - **Can't have a STB_O (strobe) without a CYC_O (cycle)**



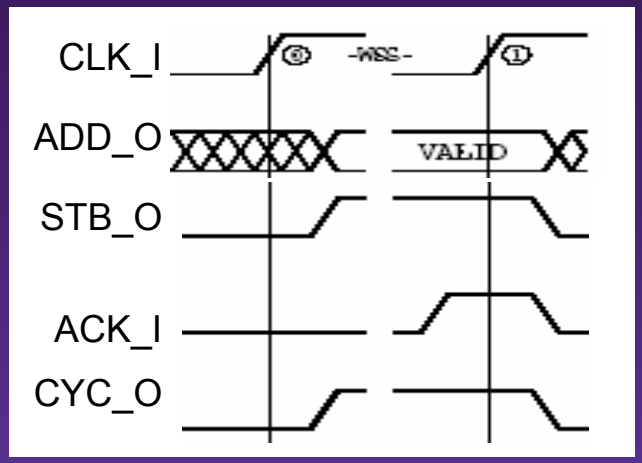

```
WB_MAS_3_25 :
assert property (
    @(posedge CLK_I)
    disable iff (!RST_I)
    STB_O |-> CYC_O )
  else $display ("VIOLATION: STB_O without CYC_O");
```

Assertion directive → assert property (
Clock edge for sampling → @(posedge CLK_I)
Asynchronous cancellation → disable iff (!RST_I)
Implication operator → STB_O |-> CYC_O)
Fail statement → else $display ("VIOLATION: STB_O without CYC_O");

Check Wishbone Master

WB_MAS_3_75_5

- **RULE 3.75.5**
  - **Whenever STB_O is asserted by the master and slave is inserting wait states then ADR_O should stay stable till ACK_I is asserted or CYC_O is de-asserted**




```
WB_MAS_3_75_5 :
   assert property (
   @( posedge CLK_I)
      disable iff ( !RST_I || !CYC_O)
      (STB_O &&  !ACK_I)  ##1 (STB_O && !ACK_I)  |->
      ( $stable( ADD_O)  [*1:$]  ##1  ACK_I ))
    else $display ( " VIOLATION: ADD_O should be stable");
```

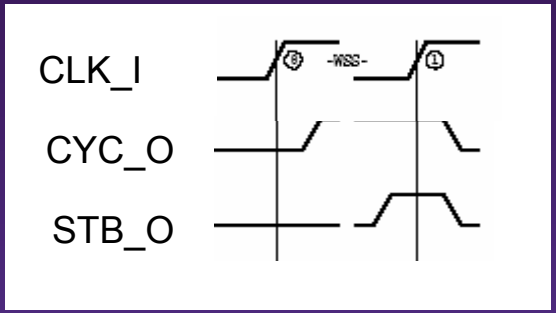
Note multiple cycles only

Sequence implication

Check Wishbone Master

WB_MAS_3_75_7

- **RULE 3.75.7**
  - **STB_O should be asserted at least once before the de-assertion of CYC_O**



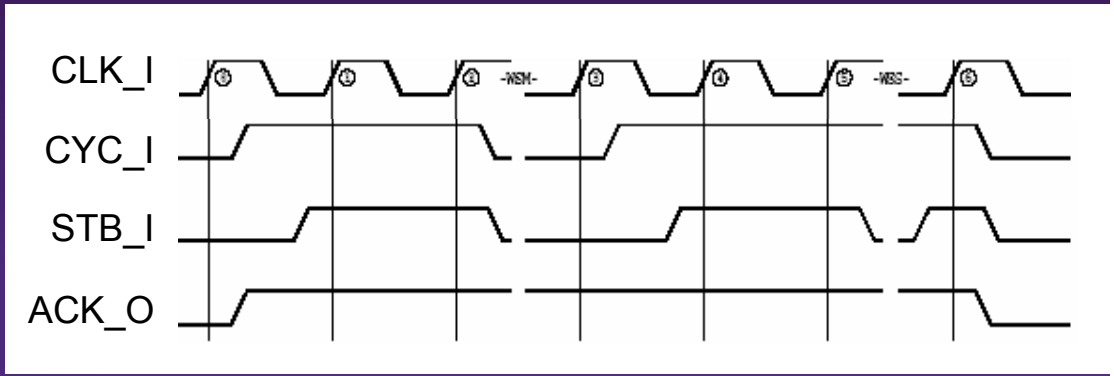

```
WB_MAS_3_75_7 :
   assert property (
     @ ( posedge CLK_I)
       disable iff (!RST_I)
       $rose (CYC_O)  |->
      (STB_O within  (  CYC_O   [*1:$] ) )  )
     else $display ( "VIOLATION: STB_O should be asserted at least once
during the window of CYC_O");
```

STB_O needs happen before (within) CYC_O goes away

Check Wishbone Slave

WB_SLA_3_75

- RULE 3.75
  - ACK_O should be asserted within max latency




```
WB_SLA_3_75 :
    assert property (
   @ (posedge CLK_I)
     disable iff (!RST_I)
       (ACK_O || !STB_I) ##1 STB_I  |->
       ##[0:latency] ACK_O )
    else $display ( " VIOLATION: The latency between STB_I and
ACK_O should not be greater than slave_latency");
```

Check Channel Arbiter

DMA Arbiter

- Verifying round-robin arbiter logic
  - No channel will be issued more than one grant while other channels have requests pending (961 checkers)
  - No grant will be issued without a request (31 checkers)
  - Only one grant is issued per clock cycle (1 checker)
  - Grant must de-assert within one clock cycle (31 checkers)
- Impended using SVA assertion library 'assert_arbiter'
  - 1024 checkers with just one instantiation
  - To use SVA Checker library include:

```
vcs -sverilog +define+ASSERT_ON –y $VCS_HOME/packages/sva
+libext+.v +incdir+VCS_HOME/packeges/sva –f vfiles
```

Check Channel Arbiter

assert_arbiter SVA checker

SVA library name

SVA Checker Library Reference Manual @ $VCS_HOME/doc/UserGuide/sva_checkerlib.pdf

```
module  channelArbiterCheck (  input logic CLK,
                               input logic RST,
                               input logic [30:0] REQ,
                               input logic [4:0]  GNT);

parameter PRI_SEL  = 3;       // Number of priorities (8)
parameter CH_COUNT = 31; // Number of channels

// ==========================================
//  request priority vector make all requests of equal priority
// ==========================================

  wire [92:0] req_priority = {93{1'b0}};

==========================================
//  Create a local grant
// ==========================================

  wire [30:0] l_gnt = (RST) ? {31{1'b0}} : ((1 << GNT) | {31{1'b0}});

==========================================================
//  Use SVA arbiter checker, set to fairness (round-robin), 31 channels
// ==========================================================

assert_arbiter # (0,              // severity
                  CH_COUNT,       // number of channels
                  PRI_SEL,        // # bits used for encoding priority
                  1,              // check only one grant is issued
                  0,              // no priority check
                  1,              // arbitration_rule (round-robin)
                  1,              //  minimum grant latency
                  0,              //  maximum grant latency
                  0,              //  clock edge_expr
                  "Arbitration Error : Channel Arbiter Error!!!!" )

arb_u1 (CLK, RST, REQ, req_priority, l_gnt);

endmodule // channelArbiterCheck module
```

options

signal port list

some customization

instance name

Check Counter Underflow

DMA Engine

- Checking that chunk size counter 'chunk_cnt' does not underflow using SVA assertion library ‘assert_no_underflow’

```
bind wb_dma_de assert_no_underflow  #(0,     // severity
                                       9,     // width
                                       0,     // min
                                      -1,     // max
                                       0,     // options
                                     "VIOLATION: chunk_cnt has underflow.")
  check_no_underflow_1 (clk, rst&&de_start , chunk_cnt);
```

design module name

assertion module name

instance name

reset

test_expr

SYNOPSYS®

Check Sequence Order

DMA Engine

- Checking that ‘write’ should follow ‘read’

R0 W0 R1 W1 - - - - Rn Wn

```
assert_write_follow_read:
   assert property (
      @(posedge clk) disable iff (!rstn)
      $rose(write) |-> $fell(read))
   else $display("VIOLATION: write asserted before read.");
```


SYNOPSYS®

Cover Property Example

Magellan Unreachable Example

- **Run assertion coverage formally**
  - **Check that CSR[`WDMA_STOP] bit controls DMA to abort**

```
cover_STOP:
  cover property (@(posedge clk)
      csr[`WDMA_STOP] ##1 (dma_abort && csr[`WDMA_ERR]));
```

  - **In WB_DMA source code:**
    - **`WDMA_STOP is 9 and ch_csr[9] is coded as 0**

```
assign  ch_csr  = CH_EN ? {9'h0, int_src_r, ch_csr_r3, rest_en, ch_csr_r2,
          ch_err, ch_done, ch_busy, 1'b0, ch_csr_r[8:1], ch_enable} : 32'h0;
```

bug

SYNOPSYS®

Dynamic Assertion Coverage Results

WB DMA/Bridge

vacuous

Assertion	Attempts	Real Success	Failure	Incomplete
test.u0.u2.check_HW_handshake.assert_ARS	1759678	0	0	0
test.u0.u2.check_HW_handshake.assert_HW_handshake	1759678	0	0	0
test.u0.u2.check_HW_handshake.assert_STOP	1759678	0	0	0
test.u0.u2.check_control_signals.assert_adr0_inc	1759678	1759677	1	0
test.u0.u2.check_control_signals.assert_adr1_inc	1759678	1759677	1	0
test.u0.u2.check_control_signals.assert_chunk_dec	1759678	1558975	200703	0
test.u0.u2.check_control_signals.assert_tot_dec	1759678	1558975	200703	0
test.u0.u2.check_control_signals.assert_write_follow_read	1759678	195252	0	0
test.u0.u2.check_external_descriptor.assert_EOF	1759678	36905	378421	0
test.u0.u2.check_external_descriptor.assert_load_nxde	1759678	3782	83597	0
test.u0.u2.check_external_descriptor.assert_write_back	1759678	22656	0	0
test.u0.u2.check_no_underflow_1.assert_no_underflow	1759678	731894	0	1
test.u0.u2.check_no_underflow_2.assert_no_underflow	1759678	329336	0	1
test.u0.u3.u0.wbmst_u1.WB_MAS_3_25	1759678	477928	0	0
test.u0.u3.u0.wbmst_u1.WB_MAS_3_75_4	1759678	106428	0	0
test.u0.u3.u0.wbmst_u1.WB_MAS_3_75_5	1759678	64540	0	0
test.u0.u3.u0.wbmst_u1.WB_MAS_3_75_6	1759678	35214	2285	0
test.u0.u3.u0.wbmst_u1.WB_MAS_3_75_7	1759678	106428	0	0
test.u0.u3.u1.wbslv_u1.WB_SLA_3_35	1759678	1575943	1	0
test.u0.u3.u1.wbslv_u1.WB_SLA_3_45	1759678	1759678	0	0
test.u0.u3.u1.wbslv_u1.WB_SLA_3_75	1759678	1759678	0	0
test.u0.u4.u0.wbmst_u1.WB_MAS_3_25	1759678	375976	0	0
test.u0.u4.u0.wbmst_u1.WB_MAS_3_75_4	1759678	93020	0	0
test.u0.u4.u0.wbmst_u1.WB_MAS_3_75_5	1759678	49340	0	0
test.u0.u4.u0.wbmst_u1.WB_MAS_3_75_6	1759678	43020	0	0
test.u0.u4.u0.wbmst_u1.WB_MAS_3_75_7	1759678	93020	0	0
test.u0.u4.u1.wbslv_u1.WB_SLA_3_35	1759678	1742269	1	0
test.u0.u4.u1.wbslv_u1.WB_SLA_3_45	1759678	1759678	0	0
test.u0.u4.u1.wbslv_u1.WB_SLA_3_75	1759678	1759678	0	0

- Assertion Coverage for DMA regression suite
  - 8 failures
    - 5 real
    - 3 time zero (false)
    - 3 vacuous
  - Does not include
    - arbiter assertions
    - Cover properties
  - Generated using VCS script assertCovReport

SYNOPSYS®

Verification Results

WB DMA/Bridge

- Dynamic Results: Found 5 failing properties
  - 4 functional RTL bugs
    - Chunk counter underflow
    - Total Size counter underflow
    - End of descriptor
    - Wishbone Master (Data not stable)
  - 1 potential bugs (RTL/Testbench)
    - Load next descriptor
- Hybrid Results: Found 5 new failing properties
  - 3 new functional RTL bugs
    - Write before Read FSM
    - STOP (cover property)
    - Automatic restart
  - 2 new potential bugs (RTL/spec)
    - Normal mode
    - Handshake mode

RTL designer needs to confirm results, not enough data available

SYNOPSYS®

Chunk Counter Bug !

RTL Code for the Counter

```
always @(posedge clk)
        if(de_start)
                chunk_cnt <= #1 txsz[24:16];
        else if(chunk_dec & !chunk_cnt_is_0_r)
                chunk_cnt <= #1 chunk_cnt - 9'h1;

assign chunk_cnt_is_0_d = (chunk_cnt == 9'h0);
always @(posedge clk)
        chunk_cnt_is_0_r <= #1 chunk_cnt_is_0_d;
```

Bug

SYNOPSYS®

Chunk Counter Bug !

RTL Code for the Counter

```
always @(posedge clk)
        if(de_start)
                chunk_cnt <= #1 txsz[24:16];
        else if(chunk_dec & (chunk_cnt>0))
                chunk_cnt <= #1 chunk_cnt - 9'h1;

assign chunk_cnt_is_0_d = (chunk_cnt == 9'h0);
always @(posedge clk)
        chunk_cnt_is_0_r <= #1 chunk_cnt_is_0_d;
```

Fix

SYNOPSYS®

Write before Read FSM Bug !

RTL Code of the state machine

```
case(state)                    // synopsys parallel_case full_case
  IDLE:  begin
                if (pause_req)                          next_state = PAUSE;
                 else if (de_start & !csr[`WDMA_ERR])
                begin  if (use_ed & !ptr_valid)     next_state = LD_DESC1;
                         else                       next_state = READ;
                end
            end
  READ:  begin
                if (dma_abort_r)                        next_state = UPDATE;
               else  if (!rd_ack)                       read = 1'b1;
                else begin
                                                        write = 1'b1;
                                                        next_state = WRITE;
                      end
            end
- - -
```

Bug

Write before Read FSM Bug !

RTL Code of the state machine

```
case(state)                    // synopsys parallel_case full_case
  IDLE:  begin
                if (pause_req)                          next_state = PAUSE;
                 else if (de_start & !csr[`WDMA_ERR])
                begin  if (use_ed & !ptr_valid)     next_state = LD_DESC1;
                         else begin  read = 1'b1;    next_state = READ; end
                end
            end
  READ:  begin
                if (dma_abort_r)                        next_state = UPDATE;
               else  if (!rd_ack)                       read = 1'b1;
                else begin
                                                        write = 1'b1;
                                                        next_state = WRITE;
                      end
             end
- - -
```

Fix

Agenda

Unit	Topic
1	Assertion Methodology
2	Design Under Test Overview
3	Design Under Test Assertion Examples
4	Assertion Coding Guidelines

General Guidelines

- Keep it simple
  - Easier to follow by other readers
  - Easier debug
  - Better error isolation when failure detected
  - Compile faster even if more numerous
  - Execute faster, especially in formal tools
- Decompose the properties into simple temporal statements
  - Joint satisfaction implies the desired design behaviors
    - One interface to next, rather than end - to - end
    - One bit of data transfer at a time (formal tools)
    - Individual causal dependencies between signals
- Use static variables rather than local variables whenever possible

Guidelines For Source Files & Development

- Source files & development:
  - Keep number of checkers per source file reasonable (simple principle)
  - Use .v or .sv file extension
  - Create filelist for SVA sources separate from HDL sources
  - Separate bindings from SVA sources and keep them in a separate SVA file for assertion reusability
  - Develop your own library with `defines and sequence/property for reuse

Guidelines For Debug

- Debugging
  - Test each assertion (or small number)
    - As they are developed (don't wait to test 10's of assertions at once)
    - Decompose assertion if needed to debug events
      - Create temporary assertions to verify sub-sequences
  - Use conditional compilation or comments to isolate assertions for debugging

Use System Tasks to Enable/Disable

Dynamic Tip 1

- **Use system tasks $assertoff/$asserton with global resets**

```
always @(negedge reset) // reset active
        $assertoff;
always @(posedge reset) // reset inactive
        $asserton;
```

‘0’ in Delay and Repetition – Caution!

Dynamic Tip 2

- *a ##0 b* **: No clock delay between** *a* **and** *b*
- *a ##0 b[*0] ##N c* **is same as** *a ##(N-1) c*
- *a ##0 b[*0] ##0 c* **has no meaning**

The |->, |=> implication – Caution!

Dynamic Tip 3

- **|->, |=> implication operators have a default "else true"**

  e.g., req |-> ack is equivalent to:

  if req then ack else 1

- **Vacuous success when antecedent is false**
- **Avoid implication with "not" operator**

```
assert_1 : assert property (@(posedge clk) not(a |=> b));
```

When "a" is false, the assertion will fail because the implication is satisfied. **The modifications for consideration**:

```
assert_1 : assert property (@(posedge clk) a |=> ! b);
```


SYNOPSYS®

Using $past Operator – Caution!

Dynamic Tip 4

Using delay for 'past' operator

- $past(data, N) is a shift register with N stages that is uninitialized and can cause syn/sim mismatches and false result
- Add delay ##N to avoid reference to uninitialized values of past operator

```
property keep_stable;
   @(posedge clk) (1'b1 ##2 !ack) |-> (data==$past(data,2));
endproperty
```

- Avoid large N in '$past' system function

Coverage Coding Guidelines

Dynamic Tip 5

- **Using *cover property* for covering significant events in the system**
- **No implication in cover property**
  - **False antecedent causes a vacuous match**

```
cov_1: cover property
    (@(posedge clk) req |=> ack);
```

  - **Correction:**

```
cov_1: cover property
    (@(posedge clk) req ##1 ack);
```

Don't Use Non-synthesizable Constructs

Formal Tip 1

- **Avoiding non-synthesizable operators, e.g., '===', '!==='**
  - **Use macro 'SVA_STD_FORMAL' to hide simulation only assertions for Magellan.**

```
`ifdef SVA_STD_FORMAL
 `else
assert_driven : assert property (
   @( posedge sva_std_clk)
            not (not_resetting && (^exp === 1'bX)));
`endif
```

  - **Magellan automatically defines 'SVA_STD_FORMAL' and users don't need to define this macro**

SYNOPSYS®

Avoid Liveness Properties

Formal Tip 2

- **Avoiding liveness properties**

```
property eventually_gnt;
   @(posedge clk) req |-> ##[4:$] gnt;
endproperty
```

  - **Simulation and Magellan never fail on liveness properties**
  - **With negated consequent cannot succeed**

```
property eventually_gnt;
   @(posedge clk) req |-> not (##[4:$] gnt);
endproperty
```

Avoid Liveness Properties

Formal Tip 2 (Cont.)

- Change the unbounded range to bounded range

```
property eventually_gnt;
  @(posedge clk) req |-> ##[4:20] gnt;
endproperty
```

- Constrain the length by intersect

```
property eventually_gnt;
  @(posedge clk) req |-> (##[4:$] gnt intersect 1[*10:20]);
endproperty
```

- Constrain the length by within

```
property eventually_gnt;
  @(posedge clk) req |-> ((##4 gnt) within  1[*10:20]));
endproperty
```


SYNOPSYS®

Large Delays Non-overlapping Transactions

Formal Tip 3

SNUG EUROPE 2005

Avoiding using large delay and repetition range

- When max_latency >20, but not overlapping transactions, use a sequential variable

```
property prop;
   @(posedge clk) disable iff (reset) $rose(req) |-> ##[1..max_latency] ack;
 endproperty
```

```
reg pre_req;
always @(posedge clk)
  pre_req <= req;

always @(posedge clk or posedge reset)
  if (reset) timer <= 0;
  else begin
     if (!pre_req && req)  timer <= 1;
     else if (timer > 0 && timer <= max_latency)  timer <= timer+1;
  end

assert_req_ack: assert property (
    @(posedge clk) ( ack |-> (timer > 0 && timer <= max_latency)));
```


SYNOPSYS®

Large Delays With Overlapping Transactions

Formal Tip 4

- When max_latency>20, and overlapping transactions:
  - Use time stamp generator (counter) and a fifo to hold the time stamp
    - See version 1 of SVA lib (VCS) ‘req_ack_unique.v’.

The Local Variable – Caution!

Formal Tip 5

- **Formal tools need RTL equivalent code**
  - **Static allocation or**
  - **Dynamic but within known fixed resources**
- **Limited support**

```
property p;
    int x;
    @(posedge clk) (a, x=data_in) |-> ##6 (data_out==x));
endproperty
```


SYNOPSYS®

The Local Variable – Caution!

Formal Tip 5 (Cont.)

- **Example of unsupported local variable usage**

```
property count_c;
    int x;
    @(clk) (a, x=0) ##1 ((c, x=x+1)[*0:$]) ##1 d |-> (x<=expr);
endproperty
```

  - **Impossible to know how many copies of local variable ‘x’ are needed between multiple overlapping attempts from this assertion**
  - **Cannot generate equivalent synthesizable RTL code for this assertion**

Verifying Data Transfers

Formal Tip 6

- Use local variable, not always efficient
- Using enumeration and static data structures – Preferred
- Verifying data using enumeration

```
genvar i;
generate
  for(i=0; i<2; i++) begin: for_all
    property p;
      @(posedge clk)
        in_ack && (i == (^ad_in)) |->
              out_ready[->1] |->  (^ad_out[7:0]) == i;
    endproperty : p
    a: assert property (for_all[i].p);
  end : for_all
Endgenerate
```

  - If DUT behavior does not depend on the specific value of ad_in, prove just for one value of i

Avoid Interaction With Action Blocks

Formal Tip 7

- **Avoid driving signals that affect assertions in action blocks**
  - **Magellan ignores action blocks and cannot see any value changes in action blocks.**

```
a : assert property (
    @(posedge clk) $rose(gnt) |-> data!=8'd0 )
     else cnt = 1'b1;
b: assert property (
    @(posedge clk) (rdy && cnt) |-> #1 !rdy);
```


SYNOPSYS®

Avoid Immediate Assertions

Formal Tip 8

- **Magellan doesn't support immediate assertions (assert in procedural code)**

```
always @(posedge clk)
   if (state==REQ )
      assert (req1 || req2)
      else …
```

Avoid Signals of Different Direction

Formal Tip 9

- **Avoiding verifying signals of different directions in a property**

Example:

```
property prop;
    @(posedge clk) start |-> ##2 (req && ack);
endproperty
```

Problem: This property cannot be used as constraint. MG cannot constrain design signals driven by design sequential logic.

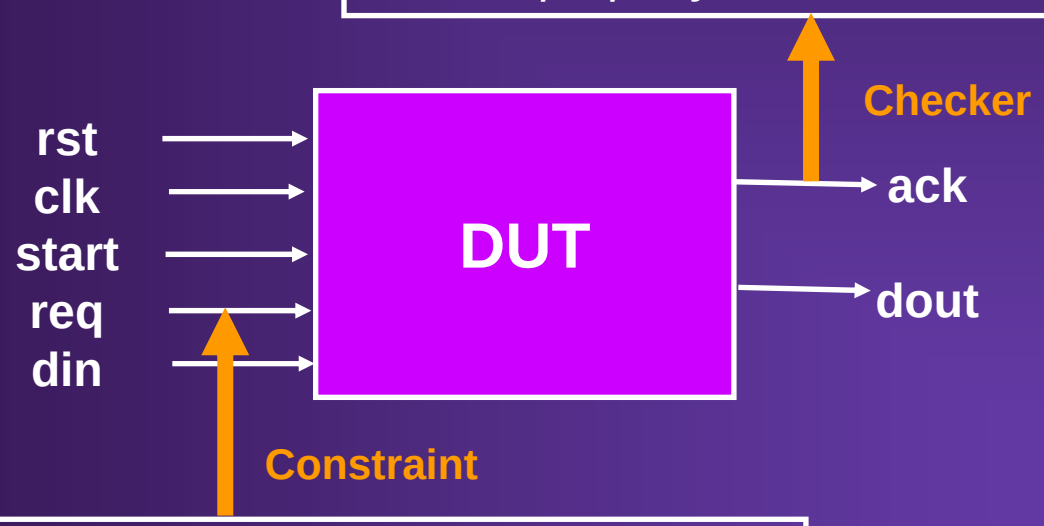
SYNOPSYS®

Avoid Signals of Different Direction

Formal Tip 9 (Cont.)

Suggestion: break 'prop' to two properties, one for input 'req' used as constraint and one for output 'ack' used as checker

```
property p2;
    @(posedge clk) start |-> ##2 ack;
endproperty
```




```
property p1;
    @(posedge clk) start |-> ##2 req;
endproperty
```

Avoid Constraining Registered Values

Formal Tip 10

- **Avoiding only referring to registered signals, s.ended, s.matched, or $past operators in constraint assertions**

```
property constraint_1;
    @(posedge clk) (stateVar1==2) |->  (stateVar2==1);
endproperty
```

Problem: this property tries to constrain the values already generated in the past.

Suggestion: use primary inputs in constraint assertions such that primary inputs can be controlled to satisfy the assertions.

Lessons Learned

Assertions Writing

- Assertions, once fully adopted as methodology
  - Should reflect the latest spec changes
  - Part of documentation
  - Better bug localization
  - Should be used to derive test plan
  - Bad specification == Bad assertion code
- Design team willing to be an active participant
  - RTL designer will need to validate results
- Assertions on interfaces are not enough
  - Low-level white-box assertions improve bug detection
  - Consider coding illegal states in SVA
- Must restrict assertion style from the beginning if planning to use formal tools

Summary

- Assertion methodology is addressing the increasing difficulty of functional verification
- Rapid advances in standard languages and new verification tools are making assertions more practical and effective
- Assertions can be reused
  - Across a broad range of verification tools: simulation, Formal, Coverage and testbench
  - From block level verification to system level verification
  - Used as checker or constraint
- Hybrid RTL formal verification can find corner case bugs missed by simulation

The End .....

Thank You !

SYNOPSYS®

Appendix-A

Verification Results

Verification Results

Wishbone Interface 8 properties

Property	Description	Result
WB_MAS_3_25	Checks that you don't get a Strobe without a Cycle	Proven
WB_MAS_3_75_4	Checks Strobe is asserted within max latency	Proven
WB_MAS_3_75_5 WB_MAS_3_75_6	Checks that address/Data remains stable when WS are inserted	Succeed Failed
WB_MAS_3_75_7	Check that strobe is asserted at least once before Cycle is de-asserted	Proven
WB_SLV_3_35	Checks if you have a termination without valid cycle	Replaymiss sim/syn mismatches
WB_SLV_3_45	Checks that only one termination is active	Replaymiss sim/syn mismatches
WB_SLV_3_75	Checks that termination occurs within max latency	Proven

Verification Results

Channel Arbiter: 1024 properties, 0 falsified and 1024 proven

Property	Description	Result
assert_arbiter_one_cycle_gnt[i]	Checks that grant signal is deasserted on next cycle	31 - Proven
assert_arbiter_granted_only_if _req[i]	Checks that there is no grant signal when there is no request	31 - Proven
assert_arbiter_single_grant	Checks that there is no more than one grant at a time	1 - Proven
assert_arbiter_two_active[i][j]	Checks that the grant on line [i] does not arrive twice without a grant on line [j] (fairness)	961 - Proven

Verification Results

WB DMA engine: 15 properties, 11 falsified and 4 proven

Property	Description	Result
assert_normal_mode	Check normal mode	Falsified
assert_HW_handshake	Check HW-handshake mode	Falsified
assert_pass_through	Check Pass-through mode	Proven
assert_ARS	Check automatic restart (ARS)	Falsified
assert_STOP	Checks that when STOP bit of csr is set, DMA immediately stops the transfer and sets ERROR bit	Proven (vacuous)
assert_EOF	Checks that when EOF bit of csr is set, DMA stops the transfer and sets DONE bit	Falsified
assert_load_nxde	Checks that when DMA finishes processing a descriptor, it will load the next descriptor	Falsified
assert_write_back	Checks that DMA will write the number of remaining bytes back to the DESC_CSR in forcing next descriptor mode	Proven
assert_write_follow_read	Checks that write should follow read	Falsified
assert_no_underflow	Checks that 'chunk_cnt' should never be underflow	Falsified
	Checks that 'tsz_cnt' should never be underflow	Falsified
assert_chunk_dec	Checks that the control signals for counter decreasing are correct	Falsified
assert_tot_dec		Falsified
assert_adr0_inc	Checks that the control signals for counter increasing are correct	Falsified
assert_adr1_inc		Proven